

MISRA C Reflection

Phillip Miavelstuck and Kabir Jolly

While working on Project 1: Traffic Signal, our group encountered some violations with regard to MISRA Compliance that we addressed.

1. MISRA23_14.4 - Addressed "Controlling expression does not have essentially Boolean type" where the following lines:

```
#if defined(DEBUG)
#elif defined(NDEBUG)
```

were then converted to be:

```
#ifdef DEBUG
#else
#ifdef NDEBUG
#endif
```

This is due to C not knowing what the boolean would be in this case, needing the other evaluation through the ifdef to determine which mode to run in.

2. MISRA23_10.3 - Addressing the issue of "Value was assigned to an object with a narrower essential type or different essential type category", `timer.tv_sec = green_light_time`; was storing a `float64_t` type variable instead of a `time_t` type variable. Although very similar, it is vital to assign it to a different type of variable. As such, the `float64_t` was first stored in a `float_64_t` variable, which was then type cast to become a `time_t` value once assigned to `timer.tv_sec`.
3. MISRA23_10.8 - Similar to the issue described above, the violation states that "The operands, 'time' and 'tv_sec', for the operator - are not of the same essential type". As such, originally, the value of time (a `float_64_t`) was being subtracted by `timer.tv_sec` of type `time_t`, where yet again, we can see that we should not mix up the types. Instead, we should utilize individual variables and type casting to get the variables to an acceptable state to be manipulated with. This was accomplished by doing the following:

```
int64_t sec_tmp = timer.tv_sec;
float64_t timer_sec_as_double = (float64_t)sec_tmp;
float64_t nsec_as_float = (time - timer_sec_as_double) * SEC_TO_NSEC;
int64_t nsec_as_int = (int64_t)nsec_as_float;
```

```
timer.tv_nsec = nsec_as_int;
```

On the other hand, there were various MISRA warnings that we intentionally ignored:

1. MISRA23_15.5 - This warning stems from multiple exit points in a function. This warning would be easy to justify suppressing even in professional environments due to it increasing the complexity of code and oftentimes complicating logic where unnecessary
2. MISRA23_13.1 - This warning was a result of pointers to values being used to initialize a struct. This warning was intentionally ignored as there was no potential for side-effects as the value that was being initialized was function-scoped and purposefully used to streamline the initialization of various global values.
3. MISRA23_10.3 - Although largely addressed throughout our code, some 10.3 warnings were ignored as casting various values to (void *) type was required to satisfy the memset function signature.
4. MISRA23_10.8 - A singular warning for this check was left in the source code. The impact of it is minor as it simply casts an integer constant into a float64_t. Other functions use similar functionality and properly save the integer constant into an intermediate variable to avoid this warning. This was an oversight; however, it has minimal impact on the program's overall functionality.

Through utilizing the Scitool Understand application to check for MISRA compliance, our group realized how important even the most minute code details are with safety-critical systems. Standards like DO-178/DO-278 require that programs are built to very rigorous standards, going far and beyond what a C/C++ compiler could check even with extreme levels of compiler warnings enabled. MISRA helps prevent subtle errors that could potentially lead to unintentional behavior and cause the program to behave unexpectedly or, in the worst case, crash. Our team plans to keep these lessons learned in mind as we continue to develop not only for future projects but also for the industry.